Name: Sabavat Sachith Naik

Roll Number: 23010103

Course: Cloud Computing

Git Hub Repository: https://github.com/Nani779-hub/

23010103-cloud-mini-project

1) Learning and Understanding: This project helped me understand how machine learning models can be trained, deployed and managed using cloud computing technologies. The main objective was to build a lightweight natural language command-parsing model using knowledge distillation and deploy it as a cloud-based service.

I learned about knowledge distillation, where a smaller student model learns from a larger teacher model. In this project, Phi-3.5-mini was the teacher and Pythia-70M was the student. The teacher generated JSON responses that were used to prepare the student's training data.

I learned how to divide data between two workers using modulo-based sharding, clean invalid records and prepare a separate validation dataset.

I also gained practical experience with Python, PyTorch and Hugging Face Transformers for model training and evaluation. SeaweedFS was used for S3-compatible storage, Docker for containerization and Kubernetes for deployment and management. I also learned FastAPI, health and readiness probes, autoscaling, subnetting, stateless NACLs and GitHub Actions for CI/CD.

Challenges Faced and Solutions

1) Teacher model connectivity :- The Kubernetes workers initially could not communicate with the Ollama Service running on my local machine.

Sol:- I configured Ollama to accept connections from the cluster and updated the worker networking settings.

2) Invalid training data :- Some teacher responses contained invalid JSON, incorrect slot values or duplicate commands.

Sol:- I implemented a cleaning pipeline. From 3000 input records 1,888 clean records were retained. I also kept validation data separate from training data.

3) Limited CPU resources :- Training and model inference were challenging because of limited computing resources.

Sol:- I used CPU-based PyTorch dependencies, a lightweight Docker image and Kubernetes resource limits.

4) Model loading and application readiness :- The service needed to download model files from storage before accepting requests.

Sol:- I configured startup and readiness probes so that Kubernetes would send traffic only when the model was ready.

5) Invalid model responses :- Sometimes the student model generated output in an incorrect JSON format.

Sol:- I added JSON validation and error handling in FASTAPI and tested the fault-injection condition.

Findings and Observations:-

The untrained Pythia-70M model achieved 0% JSON validity while trained Student v1 achieved 99% JSON validity on the 100-example validation dataset. Its full-frame exact accuracy was 27%. This showed me that valid JSON does not always mean correct information.

During the optimized Locust test, 151 requests were evaluated. 147 returned HTTP 200 and 4 returned HTTP 500, giving a success rate of 97.35%. The median latency was 2.5 seconds and p95 latency was 6.3 seconds.

I also understood the importance of Kubernetes recovery, rolling updates, load balancing and explicit inbound and outbound firewall rules.

4) Possible Improvements: In the future, I would explore model quantization or ONNX Runtime to improve CPU performance. Local model caching could reduce cold-start time. Better teacher-output validation, additional training data and more load testing could improve accuracy and reliability.

5) Conclusion: This project gave me practical experience in machine learning and cloud computing. I learned the complete process, from data preparation and model training to evaluation, Docker, Kubernetes, networking and CI/CD. It helped me understand the challenges of deploying machine-learning applications with limited resources.